🦷 Dental X-Ray Detection (YOLO) — Kaggle auto-download version

Detects **Fillings, Implant, Cavity, Impacted Tooth** on panoramic dental X-rays and adds a rule-based **treatment suggestion** layer.

What changed vs. the old notebook

- Dataset comes straight from Kaggle via `kagglehub` (no manual zip upload).
- Bigger model (YOLO11m) + larger image size, cosine LR, early stopping.
- Rare classes (Cavity, Impacted Tooth) are oversampled **in train only**.
- Boxes are validated/clipped before conversion to YOLO format.
- Final evaluation on the **test** split, plus automatic best confidence threshold.

> Runtime → Change runtime type → **GPU (T4)** before running.
> Research/education tool only. Not a clinical diagnostic device.

1. Setup

```
!pip install -q ultralytics kagglehub

import os, glob, shutil, random, yaml, cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
from PIL import Image
import torch

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CUDA available: True
GPU: Tesla T4
```

2. Download dataset directly from Kaggle

```
import kagglehub

```

```
# Download latest version
path = kagglehub.dataset_download("imtkaggleteam/dental-radiography")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'dental-radiography' dataset
Path to dataset files: /kaggle/input/dental-radiography
```

```
# Locate train / valid / test folders (each contains images + _annotat
def find_split_dirs(root):
    splits = {}
    for dirpath, dirnames, filenames in os.walk(root):
        if "_annotations.csv" in filenames:
            name = os.path.basename(dirpath).lower()
            if name in ("train", "valid", "val", "test"):
                splits["valid" if name == "val" else name] = dirpath
    return splits

SPLITS = find_split_dirs(path)
for k, v in SPLITS.items():
    print(f"{k:6s} -> {v}")

assert {"train", "valid", "test"} <= set(SPLITS), \
    "Could not find train/valid/test folders. Check the printed datase
```

```
valid  -> /kaggle/input/dental-radiography/valid
test   -> /kaggle/input/dental-radiography/test
train  -> /kaggle/input/dental-radiography/train
```

3. Explore the data

```
train_csv = pd.read_csv(os.path.join(SPLITS["train"], "_annotations.cs
train_csv.columns = [c.strip() for c in train_csv.columns]
display(train_csv.head())
train_csv.info()
```

	filename	width	height	class	xmi
0	0674_jpg.rf.e02a155a0c135687b9301ff9a20d220a.jpg	512	256	Implant	17
1	0674_jpg.rf.e02a155a0c135687b9301ff9a20d220a.jpg	512	256	Fillings	17
2	0674_jpg.rf.e02a155a0c135687b9301ff9a20d220a.jpg	512	256	Implant	22
3	0674_jpg.rf.e02a155a0c135687b9301ff9a20d220a.jpg	512	256	Implant	30
4	0674_jpg.rf.e02a155a0c135687b9301ff9a20d220a.jpg	512	256	Implant	33

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8030 entries, 0 to 8029
Data columns (total 8 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   filename  8030 non-null   object
 1   width     8030 non-null   int64 
 2   height    8030 non-null   int64 
 3   class     8030 non-null   object
 4   xmin      8030 non-null   int64 
 5   ymin      8030 non-null   int64 
 6   xmax      8030 non-null   int64 
 7   ymax      8030 non-null   int64 
dtypes: int64(6), object(2)
memory usage: 502.0+ KB
```

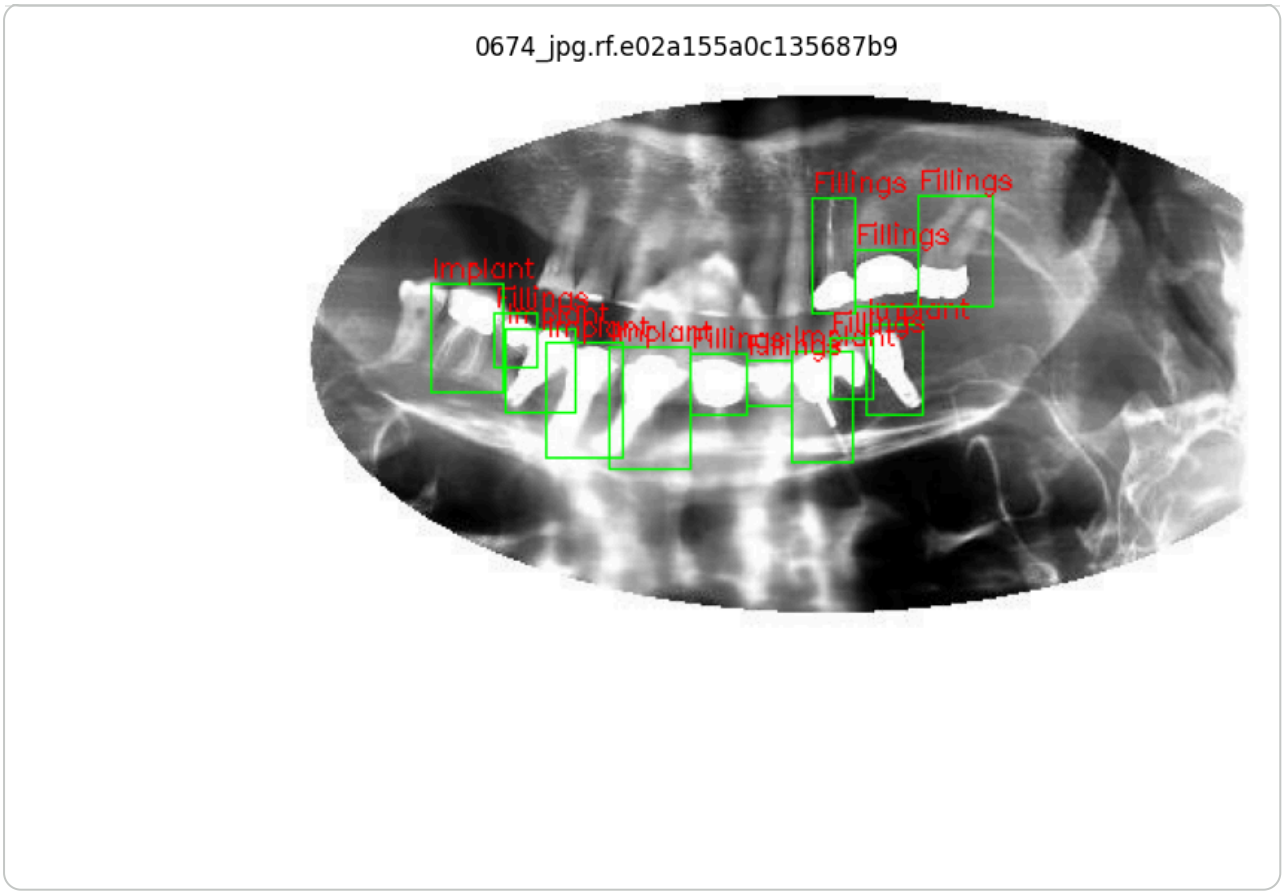
```
print(train_csv["class"].value_counts())
for s in ["train", "valid", "test"]:
    n_img = len([f for f in os.listdir(SPLITS[s]) if f.lower().endswit
    print(f"{s:6s}: {n_img} images")
```

```
class
Fillings          5242
Implant           1784
Cavity             576
Impacted Tooth     428
Name: count, dtype: int64
train : 1075 images
valid : 121 images
test  : 73 images
```

```
# Ground-truth boxes on one sample image
sample_name = "0674_jpg.rf.e02a155a0c135687b9301ff9a20d220a.jpg"
if not os.path.exists(os.path.join(SPLITS["train"], sample_name)):
    sample_name = train_csv.filename.iloc[0]

img = cv2.cvtColor(cv2.imread(os.path.join(SPLITS["train"], sample_nar
for _, r in train_csv[train_csv.filename == sample_name].iterrows():
    cv2.rectangle(img, (int(r.xmin), int(r.ymin)), (int(r.xmax), int(r
    cv2.putText(img, r["class"], (int(r.xmin), max(10, int(r.ymin) - 3
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 0, 0), 1)

plt.figure(figsize=(10, 6)); plt.imshow(img); plt.axis("off"); plt.tit
plt.show()
```



4. Convert CSV → YOLO format (with validation + rare-class oversampling)

```
CLASS_NAMES = ["Fillings", "Implant", "Cavity", "Impacted Tooth"]
CLS2ID = {n: i for i, n in enumerate(CLASS_NAMES)}

OVERSAMPLE_RARE = True   # train split only; Cavity x3, Impacted Tooth
BASE = "/content/dental_dataset"

if os.path.exists(BASE):
    shutil.rmtree(BASE)
for s in ["train", "valid", "test"]:
    os.makedirs(f"{BASE}/images/{s}", exist_ok=True)
    os.makedirs(f"{BASE}/labels/{s}", exist_ok=True)

IMG_EXT = (".jpg", ".jpeg", ".png")

def build_split(split):
    src = SPLITS[split]
    df = pd.read_csv(os.path.join(src, "_annotations.csv"))
    df.columns = [c.strip() for c in df.columns]

    unknown = set(df["class"].unique()) - set(CLASS_NAMES)
    assert not unknown, f"Unknown classes in {split}: {unknown}"

    n_img = n_box = n_skip = n_dup = 0
    for fname, g in tqdm(df.groupby("filename"), desc=split):
```

```
            src_img = os.path.join(src, fname)
            if not os.path.exists(src_img):
                continue

            lines, present = [], set()
            for _, r in g.iterrows():
                w, h = float(r.width), float(r.height)
                x1 = max(0.0, min(w, float(r.xmin))); x2 = max(0.0, min(w,
                y1 = max(0.0, min(h, float(r.ymin))); y2 = max(0.0, min(h,
                if (x2 - x1) < 2 or (y2 - y1) < 2:
                    n_skip += 1
                    continue
                cid = CLS2ID[r["class"]]
                present.add(cid)
                lines.append(f"{cid} {((x1+x2)/2)/w:.6f} {((y1+y2)/2)/h:.6
            if not lines:
                continue

            copies = 1
            if OVERSAMPLE_RARE and split == "train":
                if 2 in present:   copies += 2
                elif 3 in present: copies += 1

            stem, ext = os.path.splitext(fname)
            for k in range(copies):
                out = stem if k == 0 else f"{stem}_dup{k}"
                shutil.copy(src_img, f"{BASE}/images/{split}/{out}{ext}")
                with open(f"{BASE}/labels/{split}/{out}.txt", "w") as f:
                    f.write("\n".join(lines) + "\n")
            n_img += 1; n_box += len(lines); n_dup += copies - 1

        # images without any annotation rows -> background images (no labe
        labelled = set(df["filename"])
        for fname in os.listdir(src):
            if fname.lower().endswith(IMG_EXT) and fname not in labelled:
                shutil.copy(os.path.join(src, fname), f"{BASE}/images/{spl

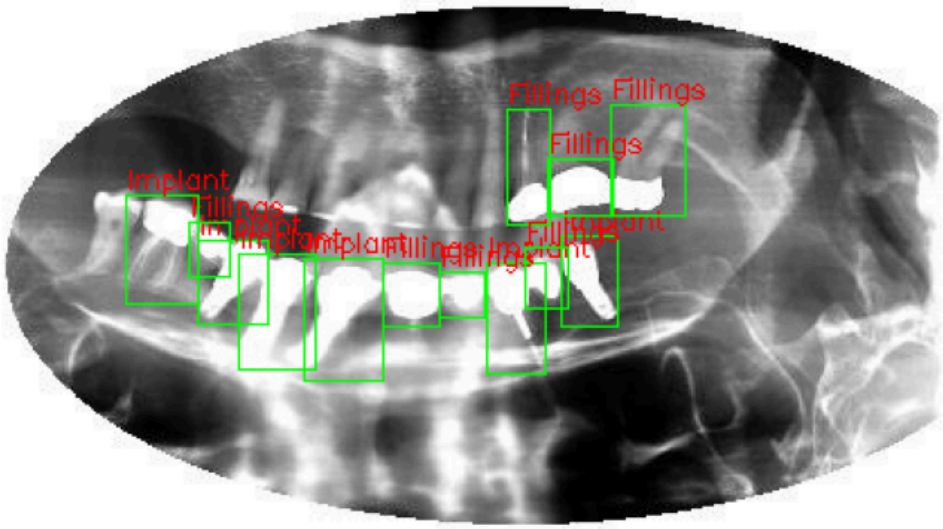
        print(f"{split}: {n_img} images, {n_box} boxes, {n_skip} degenerat

    for s in ["train", "valid", "test"]:
        build_split(s)

    for s in ["train", "valid", "test"]:
        print(s, "| images:", len(os.listdir(f"{BASE}/images/{s}")), "| la
```

```
train: 100%|██████████| 1075/1075 [00:09<00:00, 115.41it/s]
train: 1075 images, 8021 boxes, 9 degenerate boxes skipped, 671 oversar
valid: 100%|██████████| 121/121 [00:00<00:00, 188.71it/s]
valid: 121 images, 780 boxes, 0 degenerate boxes skipped, 0 oversampled
test: 100%|██████████| 73/73 [00:00<00:00, 187.00it/s]test: 73 images
train | images: 1746 | labels: 1746
valid | images: 121 | labels: 121
test | images: 73 | labels: 73
```

```
# Verify: draw YOLO labels back on an image (sanity check of the conve
lbl_path = f"{BASE}/labels/train/{os.path.splitext(sample_name)[0]}.t
img = cv2.cvtColor(cv2.imread(f"{BASE}/images/train/{sample_name}"), c
h, w = img.shape[:2]
for line in open(lbl_path):
    c, x, y, bw, bh = line.split(); c = int(c); x, y, bw, bh = map(flo
    p1 = (int((x - bw/2) * w), int((y - bh/2) * h)); p2 = (int((x + bw
    cv2.rectangle(img, p1, p2, (0, 255, 0), 1)
    cv2.putText(img, CLASS_NAMES[c], (p1[0], max(10, p1[1] - 3)), cv2.
plt.figure(figsize=(10, 6)); plt.imshow(img); plt.axis("off"); plt.sho
```




```
data_cfg = {
    "path": BASE,
    "train": "images/train",
    "val": "images/valid",
    "test": "images/test",
    "names": {i: n for i, n in enumerate(CLASS_NAMES)},
}
with open("/content/dataset.yaml", "w") as f:
    yaml.safe_dump(data_cfg, f, sort_keys=False)

print(open("/content/dataset.yaml").read())
```

```
path: /content/dental_dataset
train: images/train
val: images/valid
test: images/test
names:
  0: Fillings
  1: Implant
  2: Cavity
  3: Impacted Tooth
```

5. Train

Tuning knobs are in the first lines. If you hit **CUDA out of memory**, set `BATCH = 8` or `IMGSZ = 640`. For a faster run, use `MODEL = "yolo11s.pt"`.

```
from ultralytics import YOLO

MODEL  = "yolo11m.pt"   # yolo11s.pt = faster, yolo11l.pt = bigger
IMGSZ  = 800            # originals are 512x256; upscaling helps small
EPOCHS = 50
BATCH  = 16

model = YOLO(MODEL)

results = model.train(
    data="/content/dataset.yaml",
    epochs=EPOCHS,
    patience=20,
    imgsz=IMGSZ,
    batch=BATCH,
    optimizer="AdamW",
    lr0=0.001,
    cos_lr=True,
    warmup_epochs=3,
    weight_decay=0.0005,
    # augmentation tuned for grayscale panoramic X-rays
    hsv_h=0.0, hsv_s=0.0, hsv_v=0.3,
    degrees=5, translate=0.1, scale=0.4,
    fliplr=0.5, flipud=0.0,
    mosaic=1.0, close_mosaic=10, mixup=0.0,
    cache="ram",
    workers=2,
    seed=SEED,
    project="Dental_Project",
    name="Dental_Xray_Model",
    exist_ok=True,
    plots=True,
)
```

```
                   all        121        780      0.773      0.781

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
      45/50      12.3G     0.5415     0.2826     0.8721         14
                 Class     Images  Instances      Box(P          R
                   all        121        780      0.771      0.783

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
      46/50      12.3G     0.5287     0.2779     0.8649         23
                 Class     Images  Instances      Box(P          R
                   all        121        780      0.797      0.778

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
      47/50      12.3G     0.5273     0.2769     0.8646         10
                 Class     Images  Instances      Box(P          R
                   all        121        780      0.788      0.772

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
      48/50      12.3G     0.5166     0.2737     0.8623         14
                 Class     Images  Instances      Box(P          R
                   all        121        780      0.778      0.774

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
      49/50      12.3G     0.5175     0.2729     0.8611         27
                 Class     Images  Instances      Box(P          R
                   all        121        780      0.791      0.765
EarlyStopping: Training stopped early as no improvement observed in l
To update EarlyStopping(patience=20) pass a new patience value, i.e.

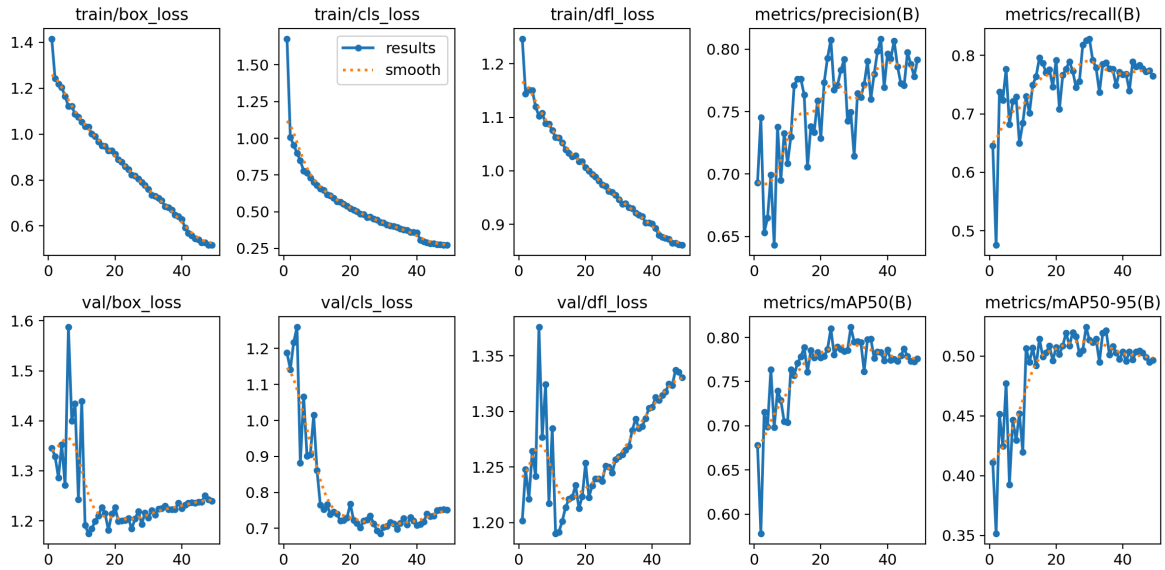
49 epochs completed in 1.202 hours.
Optimizer stripped from /content/runs/detect/Dental_Project/Dental_Xr
Optimizer stripped from /content/runs/detect/Dental_Project/Dental_Xr

Validating /content/runs/detect/Dental_Project/Dental_Xray_Model/weig
Ultralytics 8.4.198 🚀 Python-3.13.16 torch-2.11.0+cu130 CUDA:0 (Tesl
YOLO11m summary (fused): 125 layers, 20,033,116 parameters, 0 gradien
                 Class     Images  Instances      Box(P          R
                   all        121        780      0.749      0.826
               Fillings         97        540      0.868      0.889
                Implant         59        159      0.896      0.971
                 Cavity         23         43      0.564      0.698
          Impacted Tooth         25         38      0.669      0.745
Speed: 5.4ms preprocess, 19.5ms inference, 0.0ms loss, 2.9ms postproc
Results saved to /content/runs/detect/Dental_Project/Dental_Xray_Mode
```

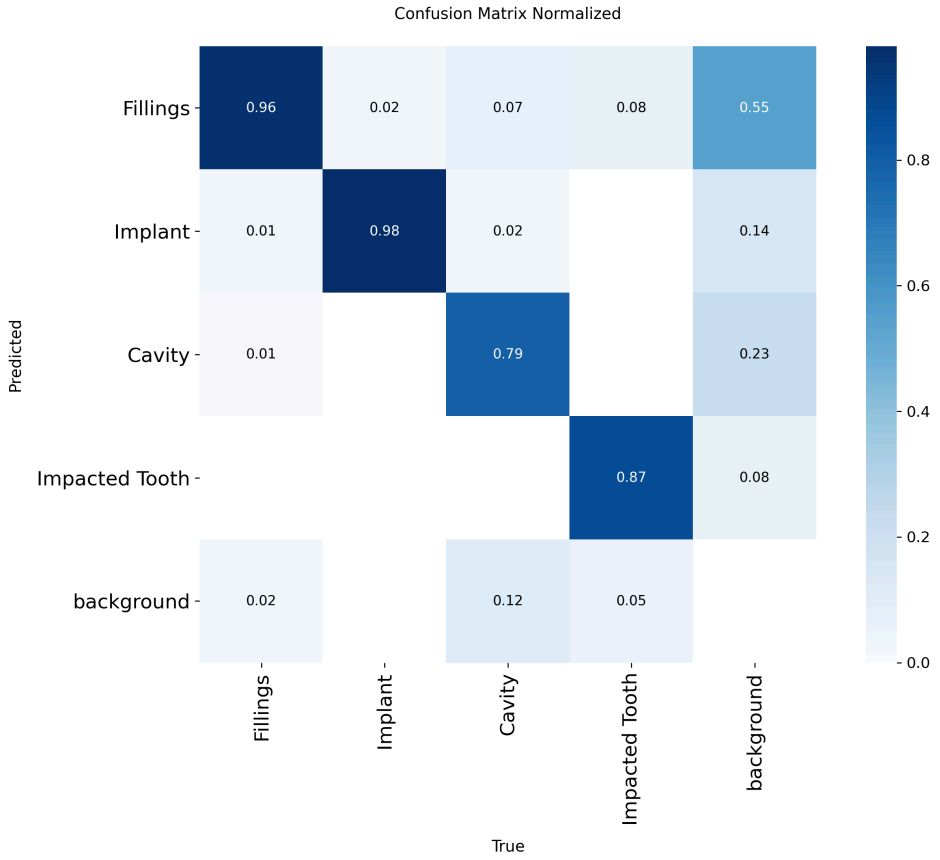
6. Training curves & confusion matrix

```
RUN_DIR = str(results.save_dir)
print("Run dir:", RUN_DIR)
display(Image.open(f"{RUN_DIR}/results.png"))
```

```
Run dir: /content/runs/detect/Dental_Project/Dental_Xray_Model
```




```
display(Image.open(f"{RUN_DIR}/confusion_matrix_normalized.png"))
```



7. Evaluate best weights on validation **and test** sets

```
best_path = f"{RUN_DIR}/weights/best.pt"
best = YOLO(best_path)
print(best.names)

print("=== VALIDATION ===")
mv = best.val(data="/content/dataset.yaml", split="val", imgsz=IMGSZ,
print(f"mAP50={mv.box.map50:.3f}  mAP50-95={mv.box.map:.3f}  P={mv.box

print("=== TEST (unseen) ===")
mt = best.val(data="/content/dataset.yaml", split="test", imgsz=IMGSZ,
print(f"mAP50={mt.box.map50:.3f}  mAP50-95={mt.box.map:.3f}  P={mt.box
```

```
{0: 'Fillings', 1: 'Implant', 2: 'Cavity', 3: 'Impacted Tooth'}
=== VALIDATION ===
Ultralytics 8.4.198 🚀 Python-3.13.16 torch-2.11.0+cu130 CUDA:0 (Tesla
YOLO11m summary (fused): 125 layers, 20,033,116 parameters, 0 gradients
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 14.8±6.7 MB/s, size
val: Scanning /content/dental_dataset/labels/valid.cache... 121 images
```

```
                   Class     Images  Instances      Box(P          R
                     all        121        780      0.748      0.825
                Fillings         97        540      0.866      0.887
                 Implant         59        159      0.896      0.972
                  Cavity         23         43      0.564      0.698
          Impacted Tooth         25         38      0.668      0.742
Speed: 0.7ms preprocess, 23.3ms inference, 0.0ms loss, 0.9ms postproces
mAP50=0.812  mAP50-95=0.524  P=0.748  R=0.825
=== TEST (unseen) ===
Ultralytics 8.4.198 🚀 Python-3.13.16 torch-2.11.0+cu130 CUDA:0 (Tesla
YOLO11m summary (fused): 125 layers, 20,033,116 parameters, 0 gradients
WARNING ⚠️ val: Slow image access detected (ping: 0.0±0.0 ms, read: 14
val: Scanning /content/dental_dataset/labels/test... 73 images, 0 backg
val: New cache created: /content/dental_dataset/labels/test.cache
                   Class     Images  Instances      Box(P          R
                     all         73        473      0.767      0.771
                Fillings         57        315      0.859      0.851
                 Implant         31        104      0.903       0.99
                  Cavity         12         22      0.468      0.591
          Impacted Tooth         17         32      0.839      0.653
Speed: 1.0ms preprocess, 23.7ms inference, 0.0ms loss, 1.0ms postproces
mAP50=0.798  mAP50-95=0.543  P=0.767  R=0.771
```

```
# Best confidence threshold from the F1 curve (validation set)
try:
    f1 = mv.box.f1_curve.mean(0)
    BEST_CONF = float(np.clip(mv.box.px[f1.argmax()], 0.15, 0.6))
except Exception as e:
    print("Could not read F1 curve:", e)
    BEST_CONF = 0.25
print("Using confidence threshold:", round(BEST_CONF, 3))
```

```
Using confidence threshold: 0.276
```

8. Predict on an image

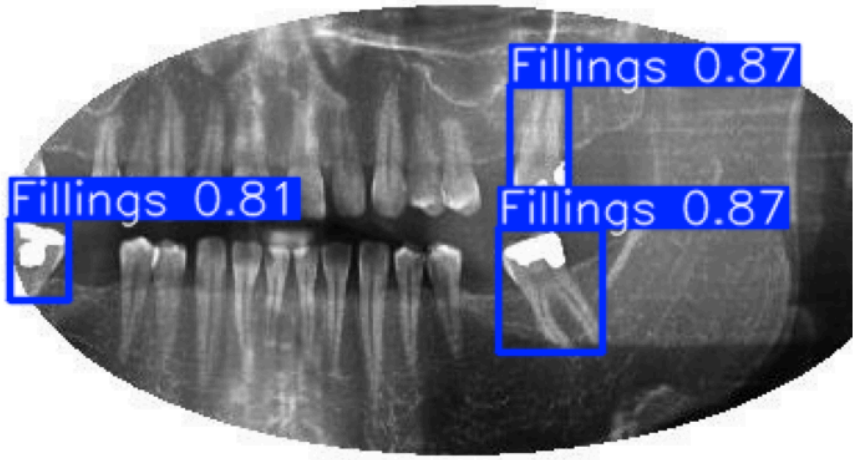
```
test_imgs = sorted(glob.glob(f"{BASE}/images/test/*.jpg"))
TEST_IMAGE = test_imgs[0]          # or set your own path
print(TEST_IMAGE)

res = best.predict(TEST_IMAGE, conf=BEST_CONF, imgsz=IMGSZ, augment=Tr

plt.figure(figsize=(12, 7))
plt.imshow(res.plot()[..., ::-1]); plt.axis("off"); plt.show()

for b in res.boxes:
    print(f"{best.names[int(b.cls[0])]:15s} : {float(b.conf[0]):.2f}")
```

```
/content/dental_dataset/images/test/0004_jpg.rf.6434bfce7667ea786e5f25
```




```
Fillings        : 0.87
Fillings        : 0.87
Fillings        : 0.81
```

9. Findings → suggested treatment (rule-based layer)

```
# The detector only finds & labels what is visible. Suggestions below a
# simple rules for decision support -- a dentist must review every resu
SUGGESTIONS = {
    "Cavity":         "Possible caries: clinical exam + restoration (fi
    "Fillings":       "Existing restoration: monitor for marginal leaka
    "Implant":        "Existing implant: check bone level and peri-impl
    "Impacted Tooth": "Impacted tooth: refer for surgical / orthodontic
}

def analyse_image(img_path, conf=None):
    conf = BEST_CONF if conf is None else conf
    r = best.predict(img_path, conf=conf, imgsz=IMGSZ, augment=True, ve
    found = {}
    for b in r.boxes:
        name = best.names[int(b.cls[0])]
        found.setdefault(name, []).append(float(b.conf[0]))
    print(f"Image: {os.path.basename(img_path)}")
    if not found:
        print("  No findings above threshold."); return r
    for name, confs in found.items():
        print(f"  - {name} x{len(confs)} (max conf {max(confs):.2f}): {
    print("  ! Decision support only. Confirm with a qualified dentist.
    return r
```